Otimização de MLP no Fashion MNIST: CPU → cuBLAS

Guilherme de Campos
guilhermedecampos12@gmail.com
Instituto Federal Catarinense - IFC
Blumenau, SC, Brasil

Mateus Dall'igna
mateusdalligna.ifc@gmail.com
Instituto Federal Catarinense - IFC
Blumenau, SC, Brasil

RESUMO

Este artigo apresenta um estudo progressivo de otimização de uma rede neural perceptron com multicamadas desenvolvida do zero em linguagem C++ para a classificação do conjunto de dados Fashion MNIST. Partindo de uma implementação sequencial executada em processador central, o modelo foi sucessivamente otimizado através de paralelização com OpenMP, desenvolvimento de funções CUDA manuais com memória persistente e, finalmente, pela utilização da biblioteca cuBLAS. Os resultados demonstram um ganho de velocidade de até 158,00x em relação à implementação original, reduzindo o tempo de treinamento de 237,0 segundos para 1,5 segundos. Adicionalmente, foi conduzida uma análise exploratória para quantificar a relação de compromisso entre o tempo de execução e a acurácia por meio de uma métrica de eficiência relativa.

PALAVRAS-CHAVE

Redes Neurais, CUDA, C++, OpenMP, cuBLAS, Otimização, Fashion MNIST

1 INTRODUÇÃO

A viabilidade do treinamento de redes neurais artificiais modernas está ligada à capacidade de processamento do hardware subjacente. Uma rede do tipo perceptron multicamadas é uma arquitetura de propagação direta composta por camadas densas, onde o fluxo de dados exige operações matriciais massivas. Matematicamente, a pré-ativação de cada camada é definida pela equação linear $z = W \cdot x + b$, que é imediatamente seguida pela pós-ativação não linear dada por $a = \sigma(z)$.

O gargalo computacional imposto pelas unidades centrais de processamento no treinamento de modelos de aprendizado atua como a principal justificativa para este estudo. A transição da carga de trabalho para arquiteturas altamente paralelas, como as unidades de processamento gráfico, tornou-se o padrão da indústria devido à escalabilidade matricial desses componentes. Por isso, compreender as etapas intermediárias de otimização de software, desde a paralelização em processadores comuns até o uso de bibliotecas de álgebra linear de baixo nível, é importante para o desenvolvimento de sistemas mais eficientes que operam com alto desempenho.

Nesse contexto, o objetivo principal deste trabalho é analisar, quantificar e comparar os ganhos de desempenho obtidos através de quatro diferentes implementações iterativas de uma mesma arquitetura de rede neural, aplicadas à classificação do conjunto de dados Fashion MNIST.

Para alcançar esse propósito, foram definidos quatro objetivos essenciais. O primeiro consiste em avaliar a eficácia do paralelismo de processador utilizando a interface OpenMP. O segundo foca em mensurar o impacto de funções personalizadas em placa de vídeo aliadas ao gerenciamento de memória persistente. O terceiro objetivo visa comparar os resultados manuais com a implementação da biblioteca padrão cuBLAS. Por fim, o quarto objetivo busca determinar o tamanho de lote ideal que ofereça o melhor balanceamento entre o tempo de convergência e a acurácia final do modelo estatístico, consolidando a análise de custo-benefício computacional.

2 FUNDAMENTAÇÃO TEÓRICA

A otimização de redes neurais em nível de hardware exige a compreensão de paradigmas de computação paralela. Enquanto o processador principal da máquina é projetado para processamento sequencial de baixa latência e controle de fluxo complexo, as placas de vídeo adotam a arquitetura de instrução única e múltiplas tarefas simultâneas, possuindo milhares de núcleos menores projetados para alto rendimento e operações matriciais independentes.

A plataforma CUDA atua como uma interface que expõe o hardware da placa de vídeo para a computação de propósito geral. O CUDA permite que desenvolvedores criem funções em linguagem C++ que são processadas simultaneamente por milhares de fluxos paralelos da placa gráfica, abstraindo o escalonamento físico dos núcleos. A literatura técnica demonstra que o maior gargalo na aceleração de algoritmos via CUDA não costuma ser o processamento aritmético em si, mas a latência na transferência de dados entre a memória do sistema e a memória de vídeo pelo barramento da placa-mãe. Para mitigar isso, bibliotecas otimizadas como a cuBLAS são empregadas por ecossistemas modernos de inteligência artificial para realizar cálculos de álgebra linear diretamente em blocos de dados mantidos exclusivamente na memória da placa de vídeo [3].

No escopo da otimização estrutural da rede, pesquisas conduzidas por [7] identificaram um fenômeno de lacuna de generalização, comprovando que redes treinadas com lotes muito grandes tendem a convergir para mínimos locais agudos na função de perda, o que degrada a capacidade do modelo de inferir dados não vistos. Simultaneamente, [8] demonstraram os benefícios empíricos do treinamento com lotes menores para maior performance do modelo, ainda que isso subutilize o paralelismo massivo do hardware. O presente trabalho complementa essa literatura ao propor uma métrica de eficiência relativa para encontrar um ponto de Pareto ótimo considerando a infraestrutura específica do teste.

3 CONJUNTO DE DADOS: FASHION MNIST

O Fashion MNIST é um conjunto de dados de referência para classificação de imagens de vestuário, elaborado como um substituto direto e mais complexo para o MNIST clássico. O conjunto contém 70.000 imagens em tons de cinza com dimensões de 28 por 28 pixels, divididas em 60.000 amostras para treinamento e 10.000 para teste, abrangendo dez categorias distintas (Tabela 1).

A escolha do Fashion MNIST justifica-se pela saturação da base original de dígitos manuscritos, onde modelos estatísticos puramente rudimentares já alcançam acurácias superiores a 99,00%, mascarando a percepção prática dos impactos e falhas arquiteturais.

Tabela 1: Classes e respectivos itens do dataset Fashion MNIST.

Classe	Item Representado	Classe	Item Representado
0	T-shirt/top	5	Sandália
1	Calça	6	Camisa
2	Suéter	7	Tênis
3	Vestido	8	Bolsa
4	Casaco	9	Bota

Para preparar os dados, os valores brutos dos pixels foram normalizados para o intervalo contínuo entre 0 e 1, dividindo-os pelo fator constante de 255,0. O vetor alvo de cada amostra foi estruturado utilizando a técnica de codificação de uma categoria ativa por vez. Todos os cálculos matemáticos deste projeto foram padronizados para utilizar precisão de ponto flutuante de 32 bits.

4 METODOLOGIA EXPERIMENTAL E ARQUITETURA

O estudo avalia quatro implementações distintas detalhadas na Tabela 2, mantendo as mesmas condições iniciais paramétricas para fins de comparação limpa e isolada do fator de hardware.

Tabela 2: Resumo das implementações testadas.

Número	Versão	Técnica Aplicada
1	Sequencial	Código C++ puro restrito a um núcleo
2	OpenMP	Paralelismo em processador de múltiplos núcleos
3	CUDA Manual	Funções customizadas na placa de vídeo
4	CUDA cuBLAS	Biblioteca proprietária otimizada na placa de vídeo

4.1 Configurações da Rede

A estrutura da rede neural empregada consiste em três camadas densas, totalizando 109.386 parâmetros treináveis. As duas camadas ocultas utilizam a função de ativação linear retificada, escolhida por sua eficiência computacional na mitigação da dissipação de gradientes. A camada de saída emprega a função de ativação sigmoide.

Os parâmetros operacionais listados na Tabela 3 foram mantidos estáticos em todas as execuções de desempenho para assegurar uma comparação simétrica. A taxa de aprendizado inicial foi fixada em 0,1, um valor adotado com base na metodologia descrita pelo estudo de He et al. [1]. O treinamento foi configurado para 10 épocas, quantidade empiricamente suficiente para a rede atingir um platô de aprendizado inicial sem gerar sobreajuste aos dados de treino, oferecendo assim uma janela de medição temporal consistente.

Para a inicialização dos pesos da rede, adotou-se o método proposto pela mesma metodologia de He et al. [1], que estabelece uma variância perfeitamente escalonada para contornar o desaparecimento de informações inerente às funções ativadoras lineares retificadas. O algoritmo de aprendizado utiliza o erro quadrático médio como métrica de perda, adequado para obter o diferencial vetorial contínuo da codificação categórica estrita.

Tabela 3: Parâmetros base fixados para o treinamento.

Parâmetro	Valor Configurado
Épocas de Treinamento	10
Tamanho do Lote Inicial	512
Taxa de Aprendizado	0,1
Função de Perda	Erro Quadrático Médio
Inicialização dos Pesos	Método de He
Inicialização das Tendências	Zeros

4.2 Ambiente Experimental

Para garantir a reprodutibilidade absoluta dos resultados e contextualizar o desempenho documentado, a Tabela 4 detalha a infraestrutura computacional utilizada de ponta a ponta.

Tabela 4: Especificações do Ambiente Experimental.

Componente	Especificação
Sistema Operacional	EndeavourOS Linux 7.0.13
Processador Central	AMD Ryzen 7 5700X3D com 8 núcleos
Memória de Processador	96 MiB de cache L3 empilhado verticalmente
Placa de Vídeo	NVIDIA GeForce RTX 4060 Ti com 8GB GDDR6
Arquitetura Gráfica	Ada Lovelace com capacidade de computação 8.9
Núcleos Gráficos	4.352 núcleos de processamento CUDA
Memória RAM	15 GiB de tecnologia DDR4
Compiladores	GCC 16.1.1 e NVCC 13.3
Versão do Driver Gráfico	610.43.02

A configuração dos sinalizadores de compilação é fundamental para a otimização de baixo nível. Para as execuções restritas ao processador central, o compilador GCC foi acionado com os atributos máximos de paralelização vetorial local. O monitoramento de duração cronometrou estritamente o laço matemático de treinamento utilizando a biblioteca nativa da linguagem C++.

4.3 Evolução das Implementações

A metodologia abrangeu a codificação sucessiva de quatro algoritmos evolutivos que resolvem a exata mesma equação de rede neural. A versão inicial sequencial foi implementada utilizando matrizes em blocos de memória contíguos para otimização de localidade de acesso rápido e multiplicadores alojados em laços de repetição simples.

Buscando segmentar o código de máquina no processador principal, a segunda iteração integrou a interface OpenMP [6]. As diretivas foram inseridas para fatiar a carga do processamento de matrizes grandes entre os dezesseis fluxos disponíveis do hardware.

A terceira iteração consistiu na migração da responsabilidade computacional para a placa de vídeo por meio de funções escritas de forma inteiramente manual. Para mitigar o gargalo de transferência no canal principal do computador, adotou-se uma arquitetura de memória persistente, onde a totalidade da memória de pesos e viés

foi transmitida e fixada uma única vez na placa de vídeo, trafegando posteriormente apenas os lotes esparsos de imagens de entrada e os valores numéricos brutos devolvidos pela verificação de perdas.

Na etapa final, os multiplicadores e somadores matemáticos gerados manualmente foram apagados da base de código e substituídos pelas rotinas de chamadas nativas da biblioteca da fabricante da placa [3], que é calibrada para manipulação de álgebra linear densa no próprio circuito do hardware de vídeo.

5 RESULTADOS E DISCUSSÃO

Para assegurar a confiabilidade e mitigar anomalias de execução nos resultados, cada uma das quatro soluções de código foi testada por um número arbitrário de 10 vezes em execuções recém-inicializadas no sistema. Todos os tempos de convergência e medidas de vazão expostos nesta documentação representam os melhores valores dentre esses testes.

A validação do salto de desempenho gerado pelas diferentes arquiteturas utilizou o conceito matemático de fator de aceleração, expresso pela razão direta entre a duração consumida pela versão elementar de controle e a duração consumida pela versão otimizada a ser validada:

$$S = \frac{T_{base}}{T_{otimizada}}$$

5.1 Comparativo de Desempenho e Fator de Aceleração

A Tabela 5 resume a extração de dados das quatro modalidades, comparando em um lote de tamanho idêntico de 512 amostras fotográficas processadas simultaneamente. A passagem sequencial pelo processador da máquina totalizou o gasto de 237,0 segundos. Com o acionamento dos paralelismos de núcleo pelo OpenMP, observou-se uma retração da contagem para 172,7 segundos, revelando uma vantagem multiplicativa de 1,37x.

A aceleração rápida da aplicação emergiu com o repasse da carga matemática para o processador gráfico. O código desenhado de forma particular pelo desenvolvedor na placa de vídeo finalizou todas as matrizes em parcos 1,7 segundos, demonstrando um aumento de 139,41x sobre o referencial do processador comum. A transcrição que utilizou a biblioteca proprietária cuBLAS manifestou a marca limítrofe de 1,5 segundos, formalizando o rendimento de 158,00x de aceleração.

Tabela 5: Resumo comparativo do desempenho das versões implementadas considerando a média de dez ensaios.

Versão Implementada	Tempo Gasto	Vazão Real	Acurácia Plena	Fator Alcançado
Sequencial	237,0 s	2.532	87,7%	1,00x
OpenMP	172,7 s	3.475	87,2%	1,37x
CUDA Manual	1,7 s	344.850	79,7%	139,41x
CUDA cuBLAS	1,5 s	394.506	78,1%	158,00x

Faz-se importante notar que as soluções executadas pela placa de vídeo manifestaram uma regressão na acurácia estatística em contraste às arquiteturas do processador principal, mostrando uma queda contínua na faixa de 8,0 a 9,6 pontos percentuais. Esse fenômeno ocorre devido a redução de quantidade de iterações realizadas, que, devido ao tamanho da batch, foi julgado necessário para uma maior velocidade de entrega.

5.2 Análise do Ponto de Equilíbrio

A pesquisa seguiu com ensaios isolados concentrado na versão cuBLAS com o objetivo de investigar qual fatia de dados repassada para a rede originava o aprendizado melhor. Para precificar o balanço simultâneo entre a rapidez do treinamento e a inteligência da rede, definiu-se a equação da eficiência relativa baseada no conceito abstrato de fronteira ótima de Pareto direcionada aos impasses de otimização multiobjetivo [10]. A métrica é estruturada como o produto da normalização do alcance da acurácia e o abatimento progressivo do tempo de processamento:

$$E(b) = \frac{A_b - A_{min}}{A_{max} - A_{min}} \times \frac{T_{max} - T_b}{T_{max} - T_{min}}$$

Onde as variáveis denotam a contagem temporal absoluta e a porcentagem de acertos para um fatiamento específico analisado, fixando-se como teto supremo 87,5% de aprendizado e admitindo como limite de rapidez a baliza de 1,2 s. O índice fracionário aponta de imediato o quanto o aumento populacional do lote auxilia a varredura da memória de vídeo sem deteriorar os gradientes da rede computacional.

Tabela 6: Resultados do experimento populacional do tamanho do lote atrelado à métrica de eficiência proposta.

Tamanho do Lote	Tempo Gasto	Acurácia	Ritmo de Vazão	Métrica Final
128	2,5 s	87,5%	240.000	0,00
256	1,9 s	81,8%	321.000	0,39
512	1,5 s	79,1%	394.000	0,60
1024	1,3 s	73,9%	450.000	0,59
2048	1,2 s	61,8%	488.000	0,31
4096	1,2 s	50,1%	515.000	0,00

A compilação de dados exposta na Tabela 6 traz à luz o comportamento visto anteriormente pela doutrina do aprendizado computacional clássico. Para partições com população estritamente inferior a 512 indivíduos, o escopo matemático deixa de alimentar adequadamente os aglomerados lógicos da placa de vídeo, depreciando o poder computacional. Alternativamente, agrupamentos excessivamente abundantes economizam apenas poucos milissegundos adicionais de passagem, mas exigem o custo de sacrificar uma porcentagem de 37 do raciocínio estatístico do modelo inferencial, reforçando a teoria da lacuna de generalização documentada por [7]. Nesse contexto, o lote de 512 amostras atingiu o valor máximo da métrica com 0,60 de eficiência prática, estabelecendo-se empiricamente como o melhor ponto de operação para a arquitetura avaliada.

6 CONCLUSÃO

A progressão de otimizações documentada neste artigo demonstrou uma redução substancial no tempo de treinamento da rede neural artificial elaborada do zero, partindo de 237,0 segundos em arquitetura sequencial para 1,5 segundos. Evidenciando um ganho prático de velocidade de 158,00 vezes através da inserção nativa e adoção instrumental da biblioteca cuBLAS, demonstrando a proficiência

das placas de vídeo frente ás dificuldades da álgebra linear imposta pelas matrizes.

O estudo evidenciou que a alocação adequada de memória e a manutenção de um estado persistente na placa de vídeo são fundamentais para explorar a capacidade de processamento paralelo das arquiteturas contemporâneas, resultado demonstrado pela aceleração de 139,41 vezes obtida na implementação manual. Além disso, identificação do tamanho de lote 512 como ponto ótimo ressalta a necessidade de balancear os ganhos temporais do hardware com a degradação preditiva causada por lotes excessivamente grandes, viabilizando a construção de modelos rápidos, porém confiáveis.

REFERÊNCIAS

[1] He, K., Zhang, X., Ren, S., e Sun, J. 2015. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 1026-1034.

[2] Zalando Research. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. Disponível em: https://github.com/zalandoresearch/fashion-mnist.

[3] NVIDIA. 2024. cuBLAS Library User Guide. Disponível em: https://docs.nvidia.com/cuda/cublas/.

[4] NVIDIA. Tensor Core Operations. Disponível em: https://developer.nvidia.com/tensor-cores.

[5] NVIDIA. 2020. TensorFloat-32 in the A100 GPU. Disponível em: https://blogs.nvidia.com/blog/2020/05/14/tensorfloat-32-precision-format/.

[6] OpenMP Architecture Review Board. OpenMP Application Programming Interface. Disponível em: https://www.openmp.org/.

[7] Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., e Tang, P. T. P. 2016. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. ICLR 2017.

[8] Masters, D., e Luschi, C. 2018. Revisiting Small Batch Training for Deep Neural Networks. arXiv:1804.07612.

[9] Smith, S. L., Kindermans, P. J., Ying, C., e Le, Q. V. 2017. Don't Decay the Learning Rate, Increase the Batch Size. ICLR 2018.

[10] Marler, R. T., e Arora, J. S. 2004. Survey of multi-objective optimization methods for engineering. Structural and multidisciplinary optimization, 26(6), 369-395.

A APÊNDICE: REPOSITÓRIO DO CÓDIGO-FONTE

O código-fonte completo com todas as implementações discutidas neste artigo, bem como os arquivos lógicos e transcrições de comandos utilizados para a extração exaustiva das métricas de desempenho abordadas ao longo da pesquisa, encontram-se disponíveis publicamente e sem restrições de uso comercial ou acadêmico no repositório digital GitHub, podendo ser acessados e replicados de forma irrestrita através do respectivo endereço eletrônico institucional e de versionamento semântico: https://github.com/MateusDallIgna/mlp-optimizations-study